SeedCraft

by hand

Format version 0 · github.com/ndssigner/seedcraft

A **SeedCraft sequence** stores a BIP-39 seed as a row of objects — beads, charms, bricks… — in a fixed order. Behind it there is a **CompactSeedQR**, the QR code of the seed that SeedSigner and NDS-Signer scan. This booklet lets you go from the sequence to that QR code, and from the QR code to a sequence, **with a pencil and nothing else**: no computer, no arithmetic. Then scan the QR code you drew with an air-gapped signer.

Before you start. Without a key, whoever has the sequence, or the QR code you draw, has the seed — even without your dictionary, if they know this method. Work alone, and burn or shred the worksheets afterwards.

Two ways to keep it. Pages 1 to 5 explain the plain way: one sequence, which **is** the seed. To keep it **with a key** — two sequences, or a sequence and a file, kept apart, each one useless alone — go to the chapter «With a key» on page 6: how to make the key with dice, how to add and subtract it, the key file, and a worked example.

Notice. SeedCraft is a mathematics exercise with a creative and artistic spirit, and it is offered as is, without any warranty. No one independent has audited it yet. If you use it with a real seed, do so on an offline computer and at your own risk. We are not liable for any loss of funds or any other harm, nor for the use anyone makes of this tool or of this method. While you use it, let no one see your screen: it shows seeds and QR codes that a camera can capture from afar.

What is in the sequence

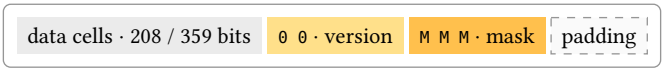
1. **The dictionary**, kept apart (a little string, a list, photos): N objects, all different. The first one means 0, the second 1, and so on up to N − 1. N is 2, 4, 8, 16 or 32.
2. **The sequence**, from its start (the clasp or a knot, or one end of an open carrier): one object per symbol. Each one stands for k bits (0s and 1s):

N (kinds of objects)	2	4	8	16	32
k (bits per object)	1	2	3	4	5
symbols, 12 words	213	107	71	54	43
symbols, 24 words	364	182	122	91	73

The bits, in order, are the QR code's **data cells** read row by row, left to right, like writing: 208 cells for 12 words (21 × 21 squares) or 359 for 24 words (25 × 25). Then come 5 header bits (next section). The bits left over at the end mean nothing.

The templates at the end of this booklet have every fixed part of the QR code already drawn — the three big squares, the dotted lines, and the 15 **format cells** that depend on the mask — so there is one template per size and mask. You only fill in the white squares: **1 = dark, 0 = leave it white**.

The header and the mask



A QR code is not drawn with its bits as they are: the standard flips some of them with one of 8 checkerboard-like patterns, the **mask**, so that the code has no large blank areas. Which one was used is written in the code's 15 **format cells**, next to its big squares. **You never apply a mask by hand**: the 3 mask bits of the header (0 to 7, see the mask table) just tell you which template page to use — its format cells are already drawn — and you fill in the data cells exactly as the bits say. The `00` before them is the format version; the padding after them (to complete the last object) is random and is ignored.

A. From a sequence to the QR code

1. **Take your dictionary** and find where the sequence starts (its clasp or knot). If you cannot tell, start at either end: if the QR code does not scan, read it again from the other end.

2. **Copy the dictionary** onto the worksheet: draw or name each object next to its number (0, 1, 2…).

3. **Write the numbers.** For each object of the sequence, write its number in the next box of the worksheet. Count the boxes: with the table above, that tells 12 words (21 × 21) or 24 words (25 × 25).

4. **Turn each number into bits** with the conversion table (column k), and write them under it.

5. **Read the header.** Count 208 bits (12 words) or 359 bits (24 words) from the start; the next five should be `00` followed by the mask's three bits (0–7). Take the template page for that size and mask.

The mask comes after the data, so it is in the **last objects** of the sequence — the last two with 8, 16 or 32 kinds of object, up to five with fewer. To draw while you read, read those last objects first, take the right page, then start from the beginning. A wrong page is not lost work: the data squares are in the same places on all eight pages of a size; only the format squares change.

6. **Fill the white squares** row by row, left to right: 1 = colour it dark (a black felt-tip works best), 0 = leave it. The number at the right of each row says how many white squares it has: tick each row off as you finish it.

7. **Scan it** with SeedSigner or NDS-Signer ("Scan a SeedQR"). One misread object is repaired by the QR code itself.

Instead of drawing, you can lay the QR code out on a fusible-bead pegboard (usually 29 × 29 pegs) or a Lego baseplate, with dark and light pieces.

B. From a SeedQR to a sequence

1. **Show the CompactSeedQR** on your signer: in SeedSigner, Backup Seed → Export as SeedQR → Compact: 21x21 or 25x25 (it shows it zone by zone so you can copy it); NDS-Signer has a SeedQR map. It must be the **Compact** kind: 21 × 21 for 12 words, 25 × 25 for 24.

2. **Find its mask.** Compare row 9 and column 9 next to its top-left big square with the drawings of the mask table: one matches. Take the template for that size and mask: its grey and black squares match the code too.

3. **Read the data cells** — the squares that are white on the template — row by row, left to right: dark = 1, light = 0. Write the bits in groups of k in the worksheet boxes.

4. **Add the header**: `00` and the mask's three bits. Complete the last group with any bits (flip a coin: zeros would make the last object easier to guess).

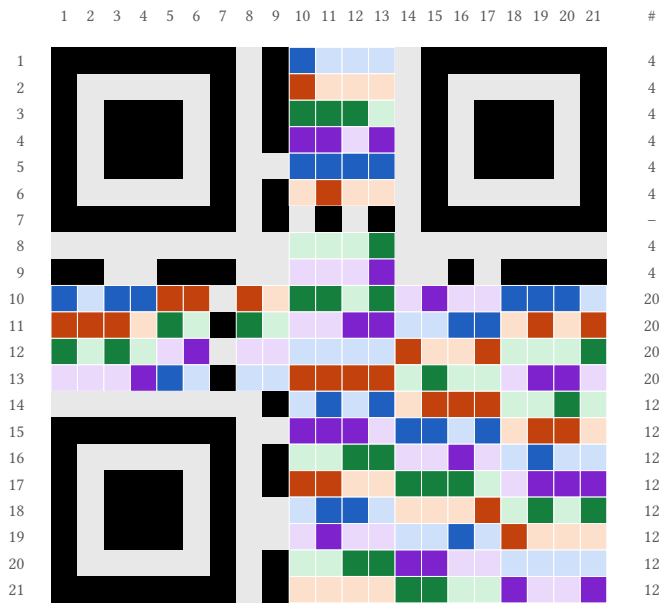
5. **Turn each group into its number** with the conversion table, and string one object per number, as your dictionary says (made beforehand, kept apart).

If your signer chose a different mask from the one SeedCraft's software would pick, your sequence is still valid: readers try every mask.

Worked example

The NDS-Signer **test seed** (public, never use it for real funds), 12 words, with N = 16 kinds of objects (k = 4: 4 bits each):

```
height demise useless trap grow lion found off key clown transfer enroll
```



Every colour is one object: its 4 data cells, read row by row, are its 4 bits — dark = 1, pale = 0. Grey and black are the template. Its CompactSeedQR uses **mask 4**, so the header is `00100` and the template page is "21 × 21 · mask 4".

The first data cells are rows 1–3, with 4 each:

row	bits	number	object
1	1 0 0 0	8	8
2	1 0 0 0	8	8
3	1 1 1 0	14	14

The end of the sequence: the last data bits, then the header and the padding.

1 0 0 1 0 0 1 0 0 0 0 0

bits 205–208 · header `00100` · padding `000`

The whole sequence, 54 objects, each in the colour of its cells. The dictionary (here, objects named 0–15) is kept apart. Check your own reading against it, five at a time:

8 8 14 13 15 4 1 1 11 14 13 4 14 14 10 3 3 5 10 4 0 9 1 1 8 15 4
6 5 7 2 14 13 6 3 2 4 12 14 7 6 1 5 4 2 8 3 12 0 0 12 9 2 0

Dictionary, kept apart: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Tables

Number → bits

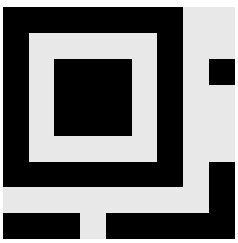
Use the column for your k. Each bit string reads left to right.

number	k = 1	k = 2	k = 3	k = 4	k = 5
0	0	00	000	0000	00000
1	1	01	001	0001	00001
2		10	010	0010	00010
3		11	011	0011	00011
4			100	0100	00100
5			101	0101	00101
6			110	0110	00110
7			111	0111	00111
8				1000	01000
9				1001	01001
10				1010	01010
11				1011	01011
12				1100	01100
13				1101	01101
14				1110	01110
15				1111	01111

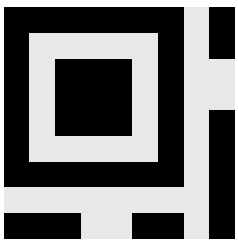
number	k = 1	k = 2	k = 3	k = 4	k = 5
16					10000
17					10001
18					10010
19					10011
20					10100
21					10101
22					10110
23					10111
24					11000
25					11001
26					11010
27					11011
28					11100
29					11101
30					11110
31					11111

Masks

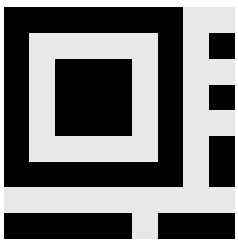
The top-left corner of a QR code (rows and columns 1–9). The 15 format cells — row 9 and column 9 — tell its mask; they are the same for 21 × 21 and 25 × 25. The header carries the mask as 3 bits.



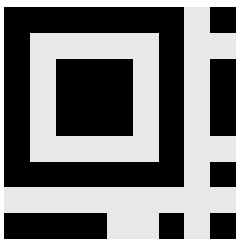
mask 0 · 000



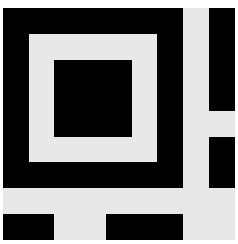
mask 1 · 001



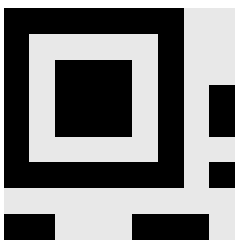
mask 2 · 010



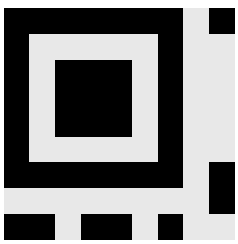
mask 3 · 011



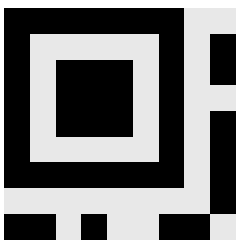
mask 4 · 100



mask 5 · 101



mask 6 · 110



mask 7 · 111

Worksheet

Destroy it when you finish. Dictionary: draw or name each object. Then one box per object of the sequence: its number above, its bits below.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120
121	122	123	124	125	126	127	128	129	130

With a key

Without a key, the sequence **is** the seed: whoever finds it and knows this method can read it. With a key it is not. The **key** is a second row of random numbers, one for each object (the specification calls it the *pad*, SPEC §10). What you make as a sequence is no longer the seed, but **the seed plus the key**, number by number, on the clock of N. The key is kept somewhere else: as a second string of objects, or written in a file.

The sequence alone says nothing about the seed, not even to someone who knows this method; **the key alone says nothing either**. Together they give the seed. That holds while the key is pure chance, secret, and used for one seed only.

Making: QR bits ⟶ number **+ key** ⟶ number ⟶ object
Reading back: object ⟶ number **− key** ⟶ number ⟶ QR bits

The rules. One key, one seed: a key used for two seeds gives both away as soon as one is known (copies of the same sequence can share its key). **Never together**: not in the same place, on the same device or the same journey. **Losing either loses the seed**: keep copies of each, each copy as far from the other half as the original. **Mark the start of each, and which is which** (a clasp, a pendant on the sequence and none on the key, a label). **Read both back together before you store them apart.**

1. Making the key by chance

The key has one number for each object of the sequence — as many as the table at the start says — each from 0 to N − 1. Its safety rests on one thing: **every number must be pure chance**, every value just as likely. Your signer or the web tool can draw it, or you can make it by hand, number by number:

- **A bag** with one object of each kind: take one without looking, note its number, **put it back** and shake the bag. If it is not put back, it is not chance. **Only if every object has the same shape, size and weight**: if they differ, some come out more often than others — use dice instead.
- **A coin**: one toss per bit, heads 1 and tails 0; k tosses per number, read with the number ⟶ bits table.
- **A die**, of any common kind. Number its faces from 0: on a die marked 1 to s, subtract 1 (a d10 marked 0–9 already is). Each throw gives the bits of its card below: keep a face in the range, written as bits with the number ⟶ bits table, and **throw again** on any other. Join the bits, first throw first, until there are k. If the last throw gives more than needed, keep its first ones.

Why throw again? A d6 has 6 faces, and 6 does not split into bits evenly: if faces 4 and 5 counted, some numbers would come out more often than others. **Example**, d6 and N = 8 (3 bits): the die shows 2 ⟶ `10`; then 5, which is thrown again; then 1 ⟶ `01`. That is `1001`; the first three, `100`, are the number **4**.

coin 1 bit per throw

heads 1, tails 0

N	2	4	8	16	32
throws	**1**	**2**	**3**	**4**	**5**

d4 2 bits per throw

keep 0–3 none thrown again

N	2	4	8	16	32
throws	**1**	**1**	**2**	**2**	**3**

d6 2 bits per throw

keep 0–3 again: 4, 5

N	2	4	8	16	32
throws	**1**	**1**	**2**	**2**	**3**

d8 3 bits per throw

keep 0–7 none thrown again

N	2	4	8	16	32
throws	**1**	**1**	**1**	**2**	**2**

d10 3 bits per throw

keep 0–7 again: 8, 9

N	2	4	8	16	32
throws	**1**	**1**	**1**	**2**	**2**

d12 3 bits per throw

keep 0–7 again: 8–11

N	2	4	8	16	32
throws	**1**	**1**	**1**	**2**	**2**

d16 4 bits per throw

keep 0–15 none thrown again

N	2	4	8	16	32
throws	**1**	**1**	**1**	**1**	**2**

d20 4 bits per throw

keep 0–15 again: 16–19

N	2	4	8	16	32
throws	**1**	**1**	**1**	**1**	**2**

Never numbers you choose yourself, nor taken from the seed, a password or a date. A key made by hand can also be added, on the clock, to one drawn by your signer: the result is safe as long as either of the two is.

2. Adding on the clock (making)

The numbers add up on a **clock of N**: after N − 1 comes 0 again. For each object, the number from the QR code plus the number of the key; if the result is N or more, subtract N. The addition table does it for you: **the row of the QR's number, the column of the key's number**; where they cross is the number of the object you make.

Example, N = 8: the QR gives 6 and the key says 7. 6 + 7 = 13, which is 8 or more: 13 − 8 = **5**. In the table for 8, row 6, column 7: **5** (shaded).

3. Subtracting on the clock (reading back)

To read it back, the number of the object minus the number of the key; if the result is below 0, add N. With the same table, the other way round: **go to the key's column, look down it for the object's number**; its row is the number from the QR.

Example, N = 8: the object is 5 and the key 7. 5 − 7 = −2, below 0: −2 + 8 = **6**. In the table for 8, column 7, the 5 is in row **6**.

N = 8

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

N = 2

+	0	1
0	0	1
1	1	0

N = 4

+	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

Addition tables. Making: row = number from the QR, column = key, cell = object. Reading back: in the key's column, find the object; its row is the number from the QR. The table for 32 is on the next page.

N = 16

+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0
2	2	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1
3	3	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2
4	4	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3
5	5	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4
6	6	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5
7	7	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6
8	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7
9	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8
10	10	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9
11	11	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10
12	12	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11
13	13	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12
14	14	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13
15	15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Addition on the clock of 32. Row = number from the QR, column = key, cell = object. Reading back: in the key's column, find the object; its row is the number from the QR.

+	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0
2	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1
3	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2
4	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3
5	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4
6	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5
7	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6
8	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7
9	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8
10	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9
11	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10
12	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11
13	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12
14	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13
15	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
16	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
17	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
18	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
19	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
20	20	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
21	21	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
22	22	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
23	23	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
24	24	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
25	25	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
26	26	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
27	27	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
28	28	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
29	29	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
30	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
31	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

4. The key as a file

Instead of a second string, the key can be a text file that a person can read and the web tool can open:

```
71 8
0567 5511 7724 2572 4
7120 5374 5561 5153 0
6754 7077 3113 1030 0
3275 7212 6501 2400 1
4220 035 3
```

The key of the worked example (N = 8). **An example only: never use a key taken from a document.**

- **First line**: how many numbers there are, and N.
- **Then the key**: 16 numbers per row, in groups of 4, one character each.
- **The last character of each row is a check.** If a number is copied wrong, the check of its row no longer matches. It comes from the key alone and says nothing about the seed.
- Lines that start with `#` are notes. The file does not say what it is: do not call it "seed".

Each number is one character: `0` to `9`, then `A` for 10, `B` for 11, and so on up to `V` for 31:

number	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
character	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
number	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
character	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**	**S**	**T**	**U**	**V**

The check character, by hand

Start at the row's **last** number (not counting the check) and go leftwards. **Double every other one**, starting with that last one; if a double is more than N − 1, subtract N − 1. Add everything up. The check is what is missing to reach the next multiple of N (0 if the sum already is one). It catches any single number copied wrong, and almost any two neighbours swapped (not 0 and N − 1; with N = 2 no swaps at all).

The first row of the example, N = 8 (doubles above 7 lose 7):

number	0	5	6	7	5	5	1	1	7	7	2	4	2	5	7	2
double?	–	×2	–	×2	–	×2	–	×2	–	×2	–	×2	–	×2	–	×2
counts	**0**	10 − 7 = **3**	**6**	14 − 7 = **7**	**5**	10 − 7 = **3**	**1**	**2**	**7**	14 − 7 = **7**	**2**	8 − 7 = **1**	**2**	10 − 7 = **3**	**7**	**4**

Sum: 0 + 3 + 6 + 7 + 5 + 3 + 1 + 2 + 7 + 7 + 2 + 1 + 2 + 3 + 7 + 4 = **60**. The next multiple of 8 is 64, so the check is 64 − 60 = **4**, written **4** — the last character of the row.

Worked example with a key

The same **test seed** (public, never use it for real funds), now with N = 8 kinds of objects (k = 3: 3 bits each), so 71 objects. Its CompactSeedQR uses **mask 4**. The key is the one of the key file (section 4): **a public example key, never use it**.

Making: the first four objects

The QR's data cells, row by row, 3 bits per object; cells as row·column.

object	QR cells	bits	number	+ key	sum	object
1	1·10, 1·11, 1·12	`100`	4	0	4 + 0 = 4	**4**
2	1·13, 2·10, 2·11	`010`	2	5	2 + 5 = 7	**7**
3	2·12, 2·13, 3·10	`001`	1	6	1 + 6 = 7	**7**
4	3·11, 3·12, 3·13	`110`	6	7	6 + 7 = 13	13 − 8 = **5**

Reading back: the same four

object	its number	− key	= number	bits	QR cells
1	4	0	4 − 0 = 4 ⟶ **4**	`100`	1·10, 1·11, 1·12
2	7	5	7 − 5 = 2 ⟶ **2**	`010`	1·13, 2·10, 2·11
3	7	6	7 − 6 = 1 ⟶ **1**	`001`	2·12, 2·13, 3·10
4	5	7	5 − 7 = −2 ⟶ −2 + 8 = **6**	`110`	3·11, 3·12, 3·13

The mask first

By hand you need the template page before you draw, and the mask is in the last objects: subtract those first. Here the last two:

object	its number	− key	= number	bits
70	7	3	7 − 3 = 4 ⟶ **4**	`100`
71	1	5	1 − 5 = −4 ⟶ −4 + 8 = **4**	`100`

Bits 208–210: the last data cell (21·21) and the version `00`; then the mask, `100` = **4**: the page "21 × 21 · mask 4".

All 71, to check your work

object	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
QR	4	2	1	6	6	7	6	4	0	4	3	3	7	3	2	4	7	3	5	2	1	4	6	5
+ key	0	5	6	7	5	5	1	1	7	7	2	4	2	5	7	2	7	1	2	0	5	3	7	4
= sequence	**4**	**7**	**7**	**5**	**3**	**4**	**7**	**5**	**7**	**3**	**5**	**7**	**1**	**0**	**1**	**6**	**6**	**4**	**7**	**2**	**6**	**7**	**5**	**1**

object	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
QR	5	1	0	0	4	4	2	1	4	3	6	4	3	1	2	7	1	3	5	5	3	0	6	2
+ key	5	5	6	1	5	1	5	3	6	7	5	4	7	0	7	7	3	1	1	3	1	0	3	0
= sequence	**2**	**6**	**6**	**1**	**1**	**5**	**7**	**4**	**2**	**2**	**3**	**0**	**2**	**1**	**1**	**6**	**4**	**4**	**6**	**0**	**4**	**0**	**1**	**2**

object	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
QR	2	3	1	6	3	5	4	1	2	5	0	2	4	0	7	4	0	0	1	4	4	4	4
+ key	3	2	7	5	7	2	1	2	6	5	0	1	2	4	0	0	4	2	2	0	0	3	5
= sequence	**5**	**5**	**0**	**3**	**2**	**7**	**5**	**3**	**0**	**2**	**0**	**3**	**6**	**4**	**7**	**4**	**4**	**2**	**3**	**4**	**4**	**7**	**1**

Worksheet with a key

Destroy it when you finish, and never keep it with the key or the sequence. One line per object. Making: QR number + key = result, then the object for the result. Reading back: object, its number (result) − key = QR number. Use a second sheet for more than 96.

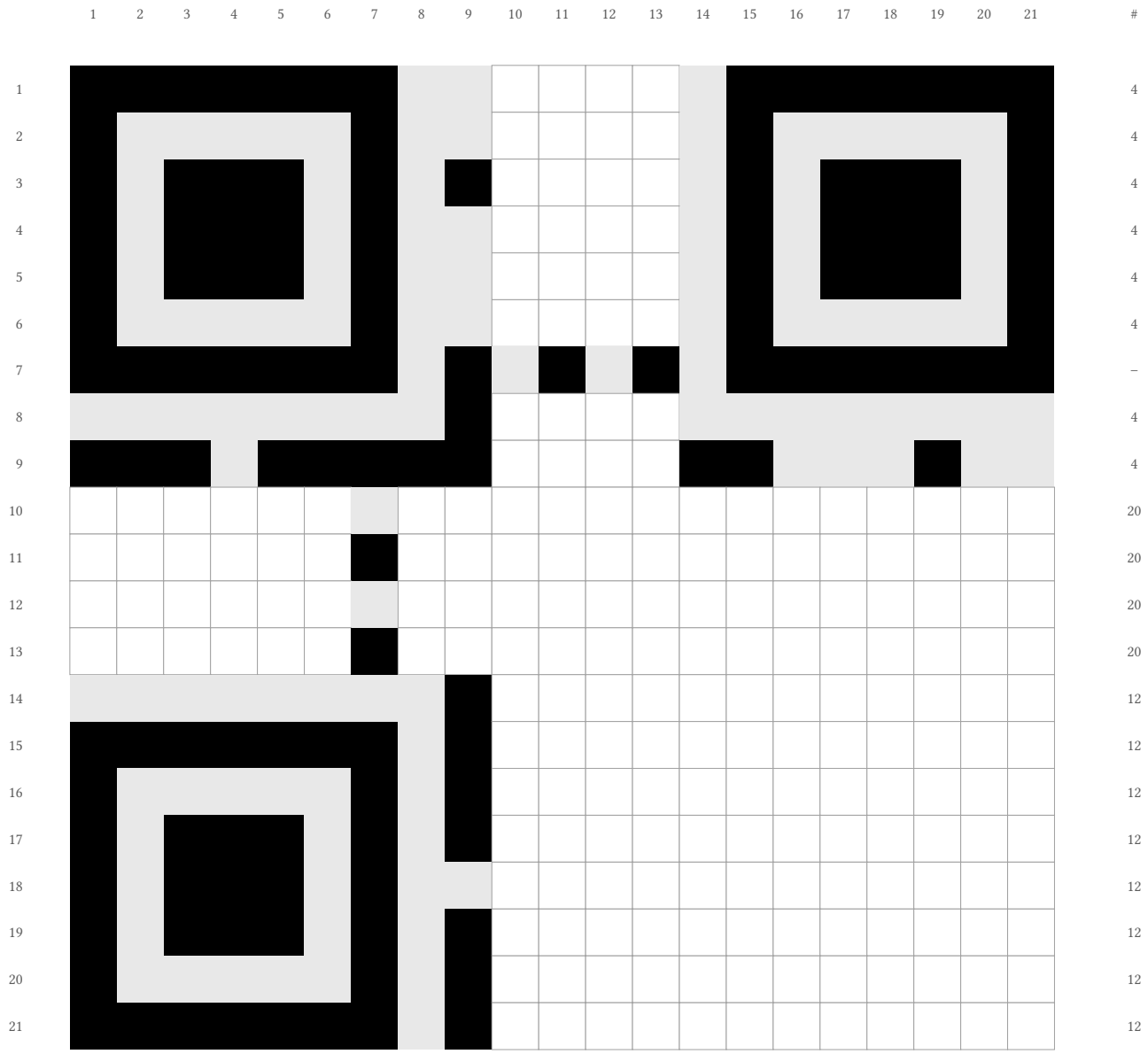
#	QR	key	result	object
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				
32				

#	QR	key	result	object
33				
34				
35				
36				
37				
38				
39				
40				
41				
42				
43				
44				
45				
46				
47				
48				
49				
50				
51				
52				
53				
54				
55				
56				
57				
58				
59				
60				
61				
62				
63				
64				

#	QR	key	result	object
65				
66				
67				
68				
69				
70				
71				
72				
73				
74				
75				
76				
77				
78				
79				
80				
81				
82				
83				
84				
85				
86				
87				
88				
89				
90				
91				
92				
93				
94				
95				
96				

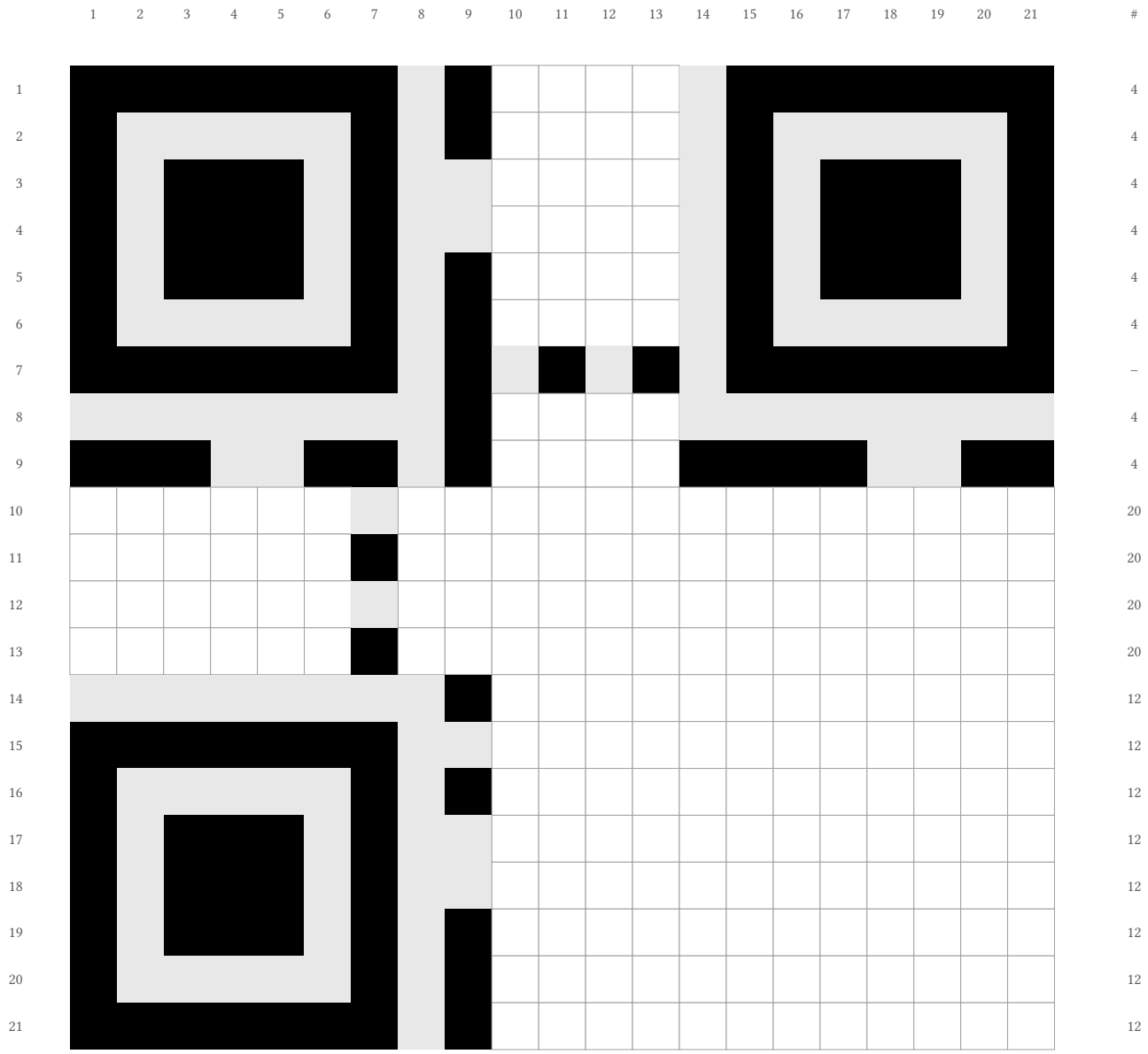
Template 21 × 21 · mask 0

12 words · header `00000` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



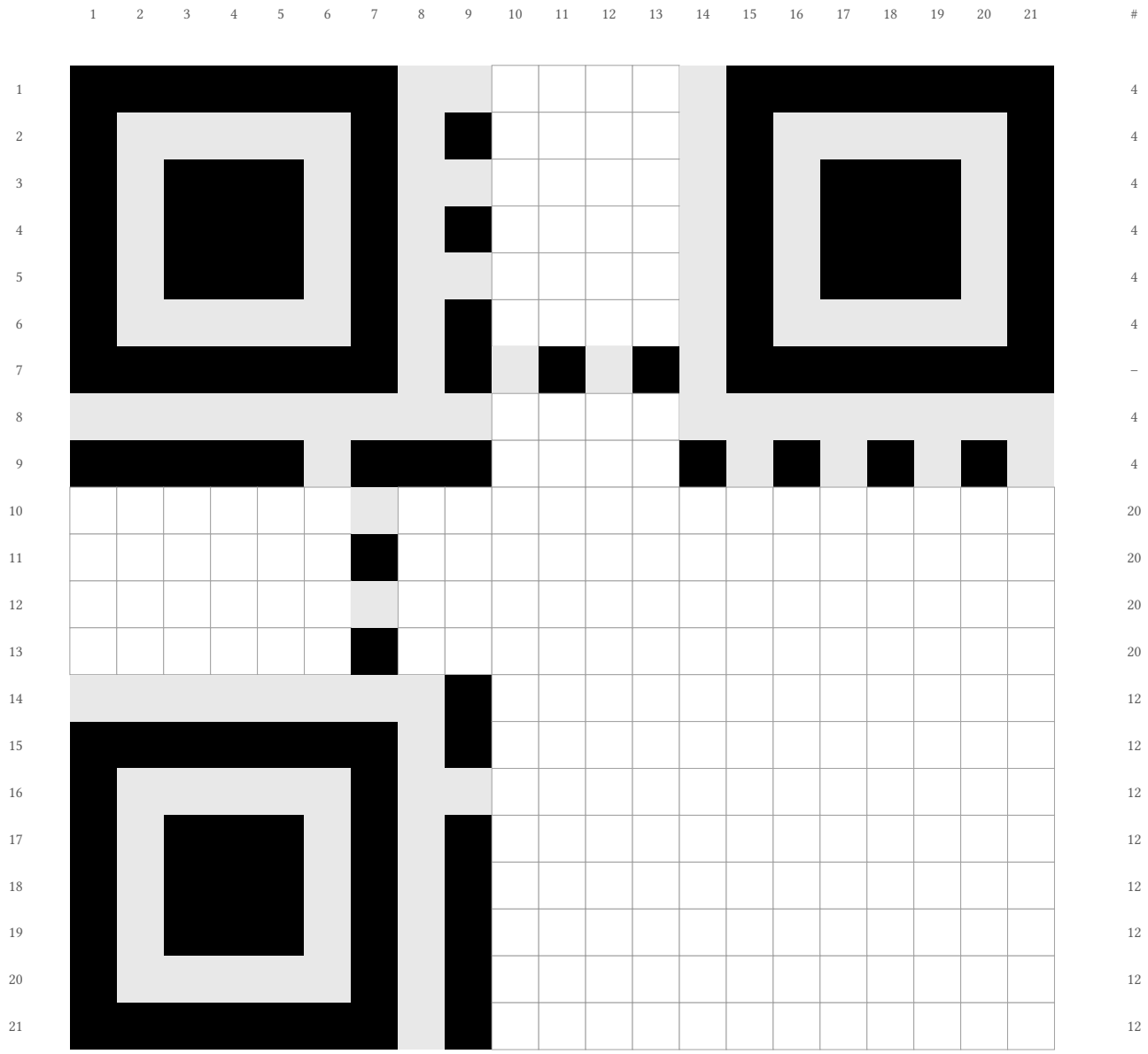
Template 21 × 21 · mask 1

12 words · header `00001` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



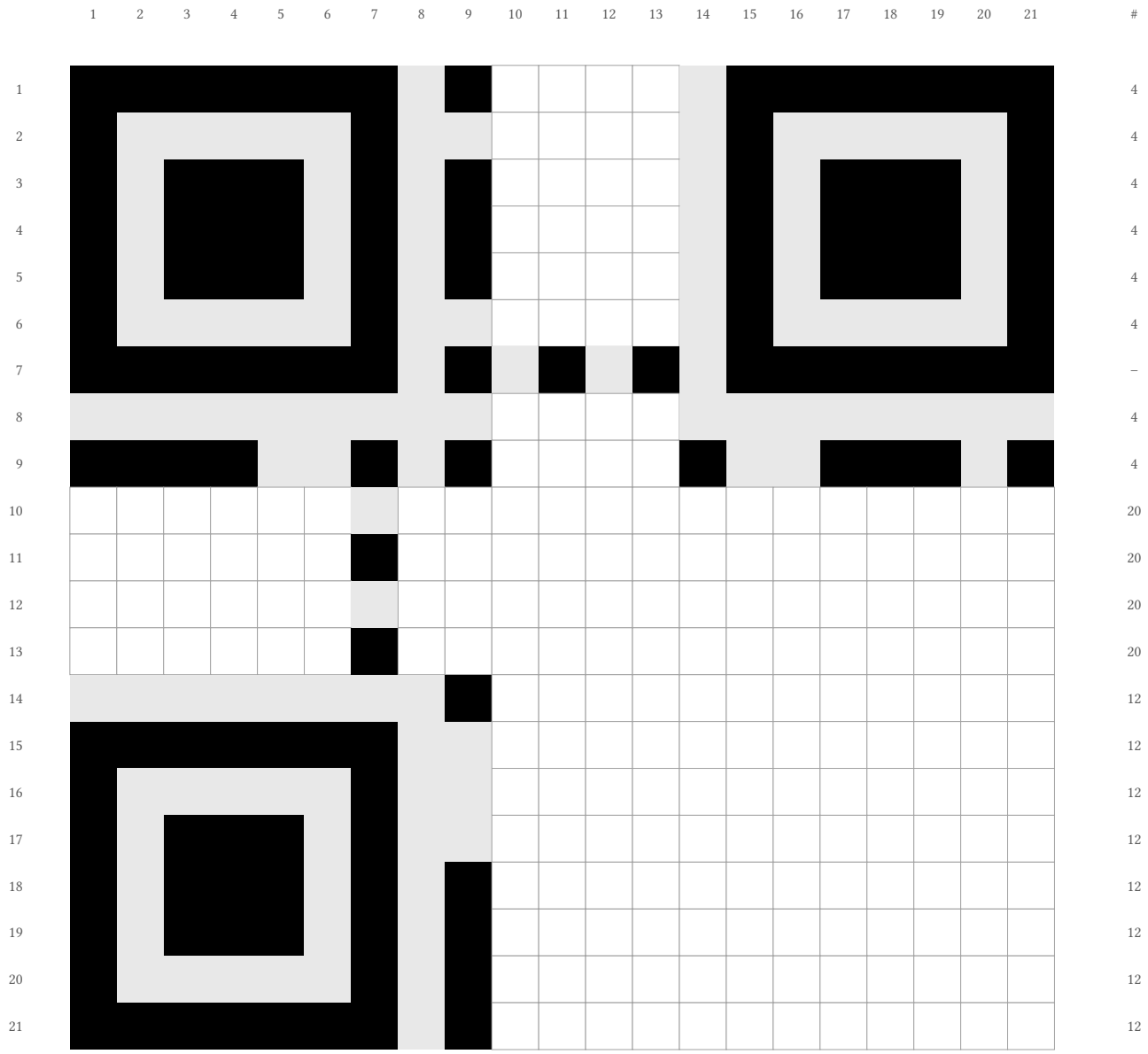
Template 21 × 21 · mask 2

12 words · header `00010` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



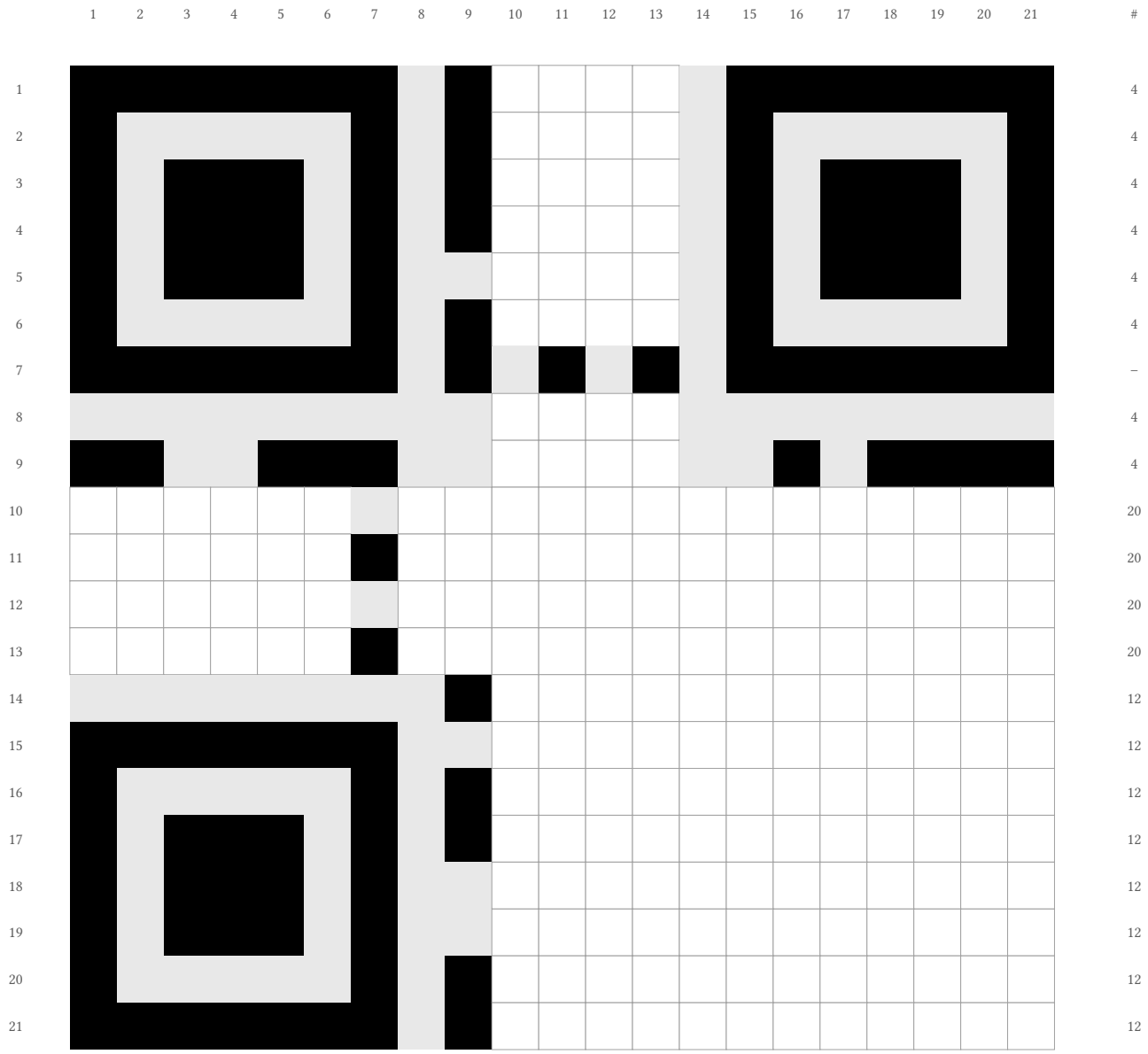
Template 21 × 21 · mask 3

12 words · header `00011` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



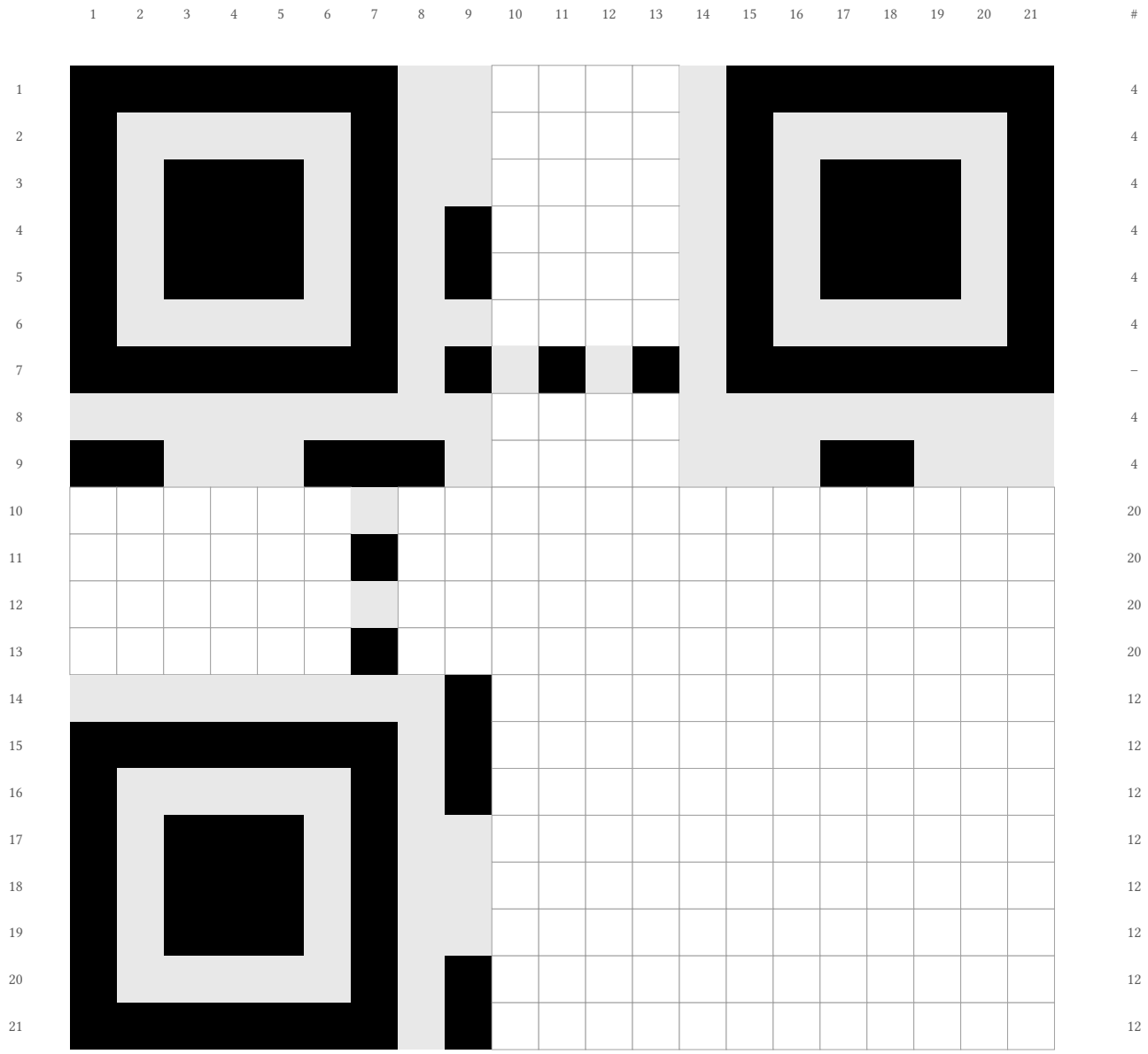
Template 21 × 21 · mask 4

12 words · header `00100` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



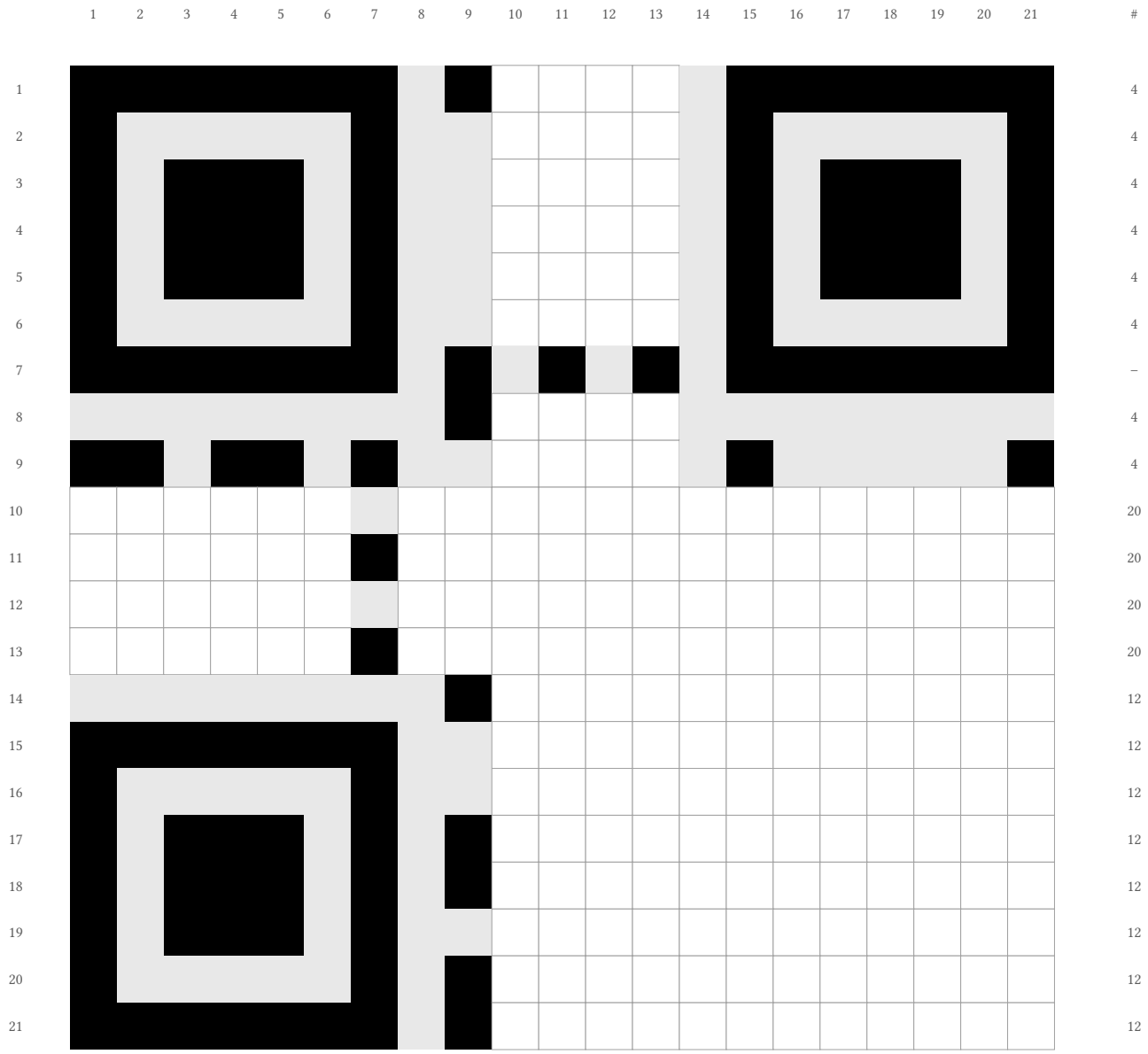
Template 21 × 21 · mask 5

12 words · header `00101` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



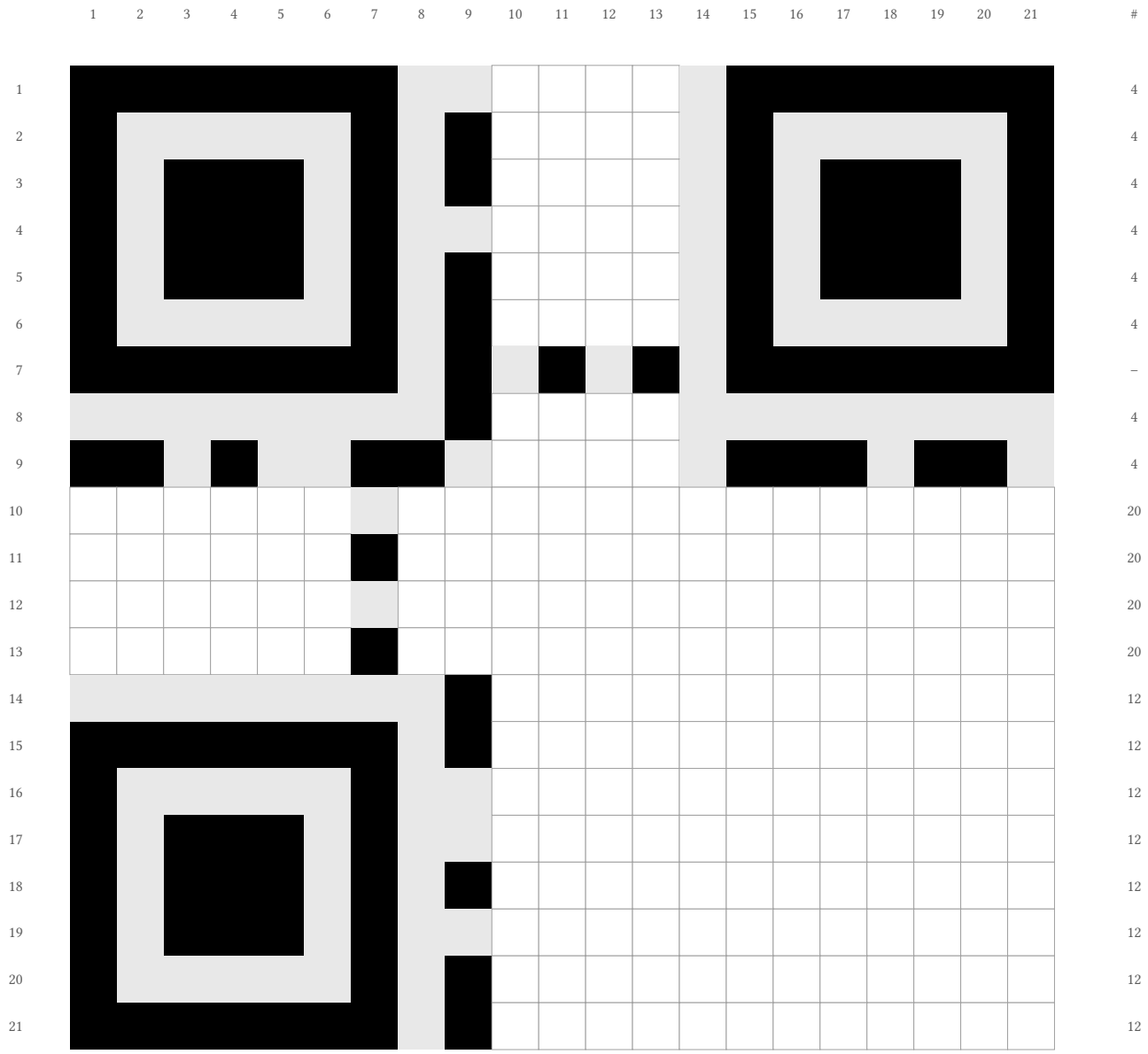
Template 21 × 21 · mask 6

12 words · header `00110` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



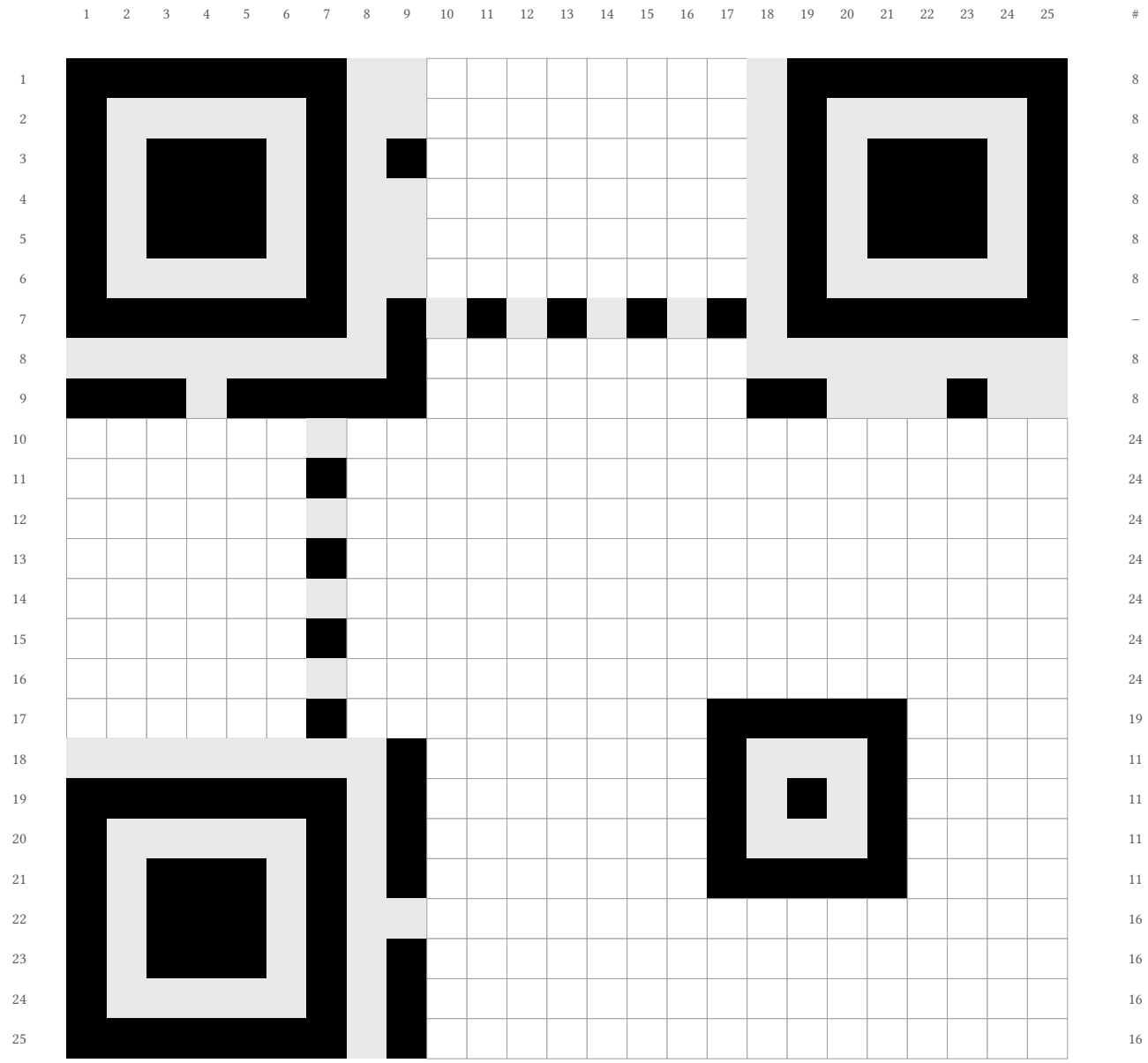
Template 21 × 21 · mask 7

12 words · header `00111` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



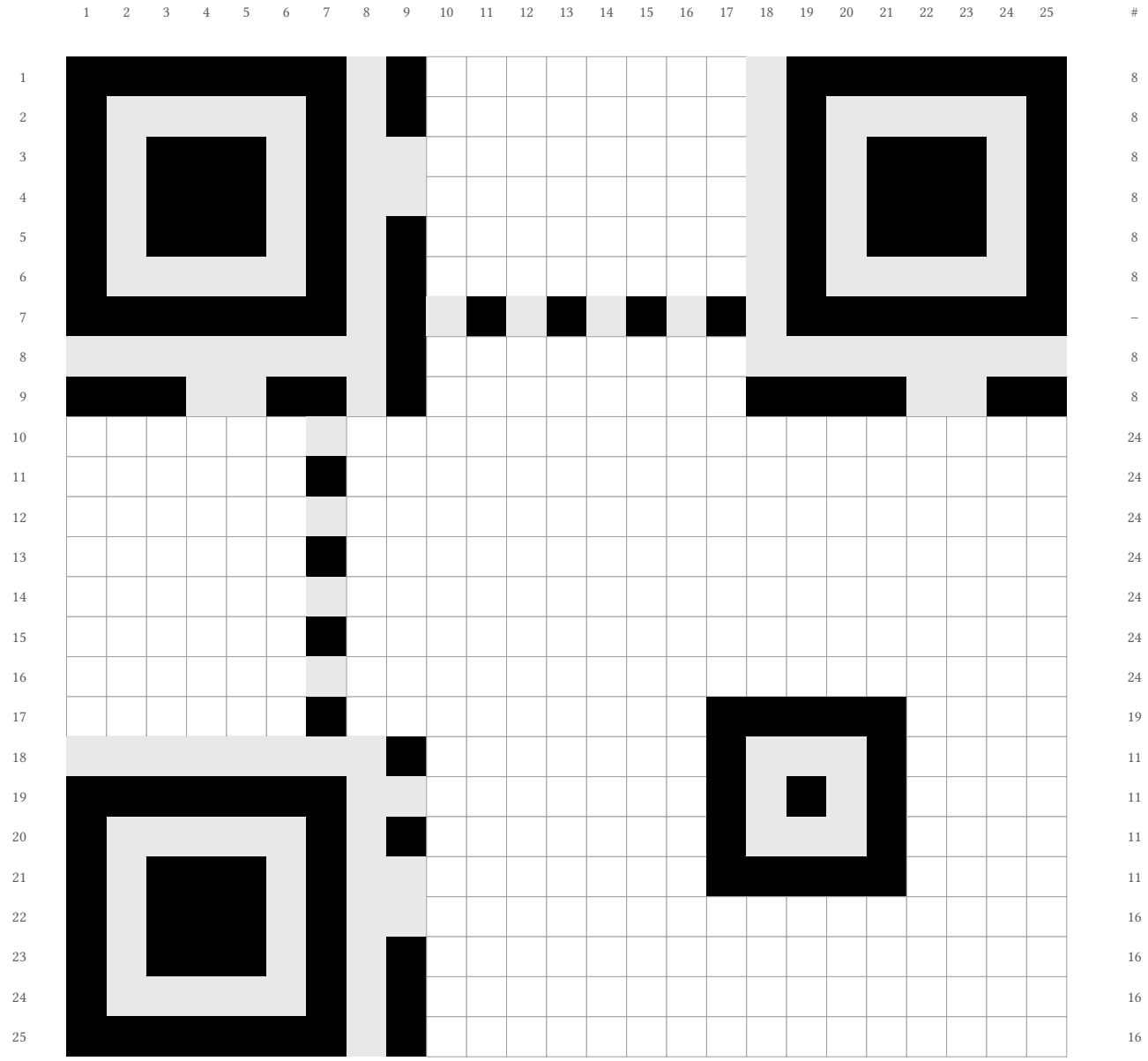
Template 25 × 25 · mask 0

24 words · header `00000` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



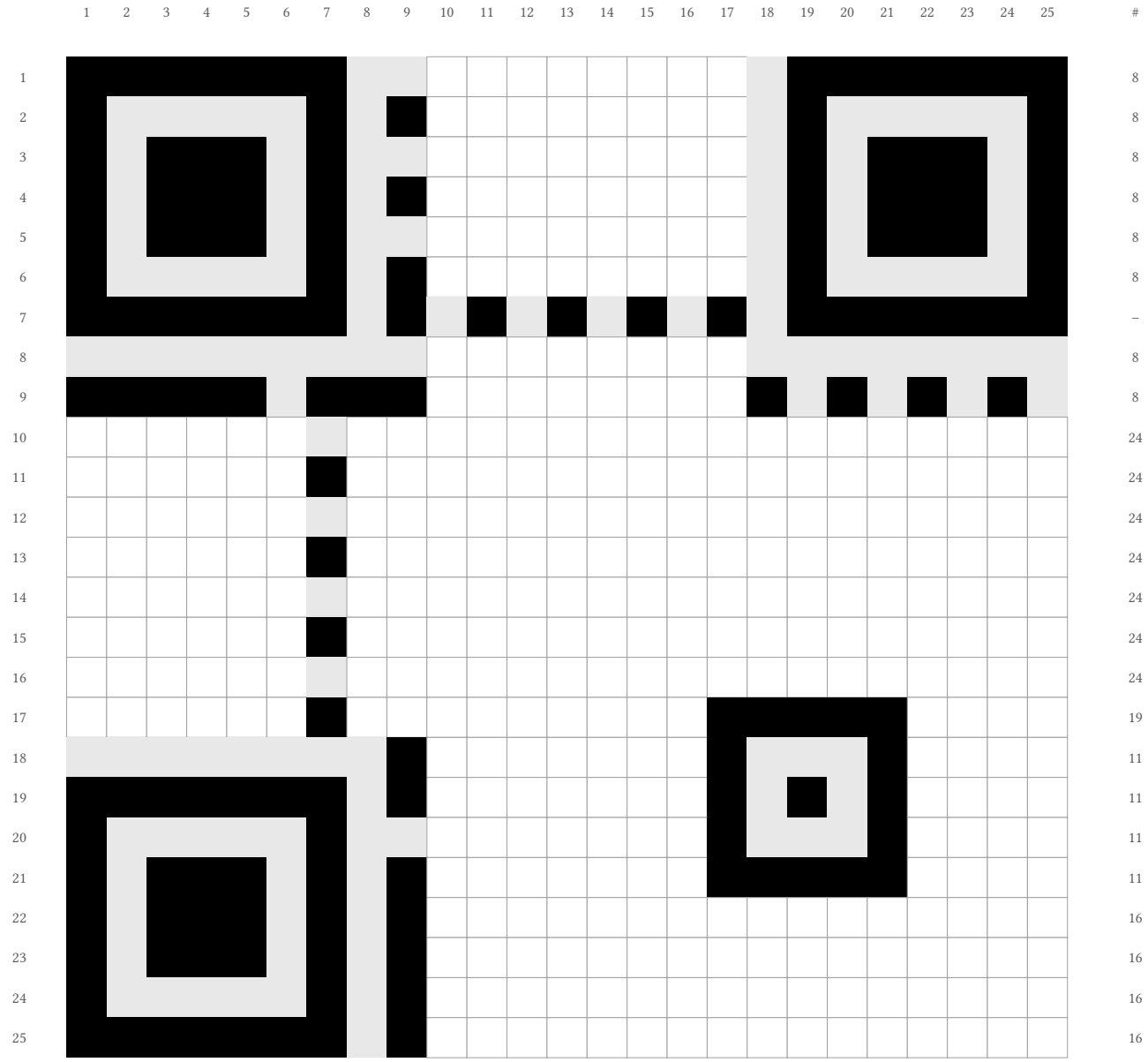
Template 25 × 25 · mask 1

24 words · header `00001` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



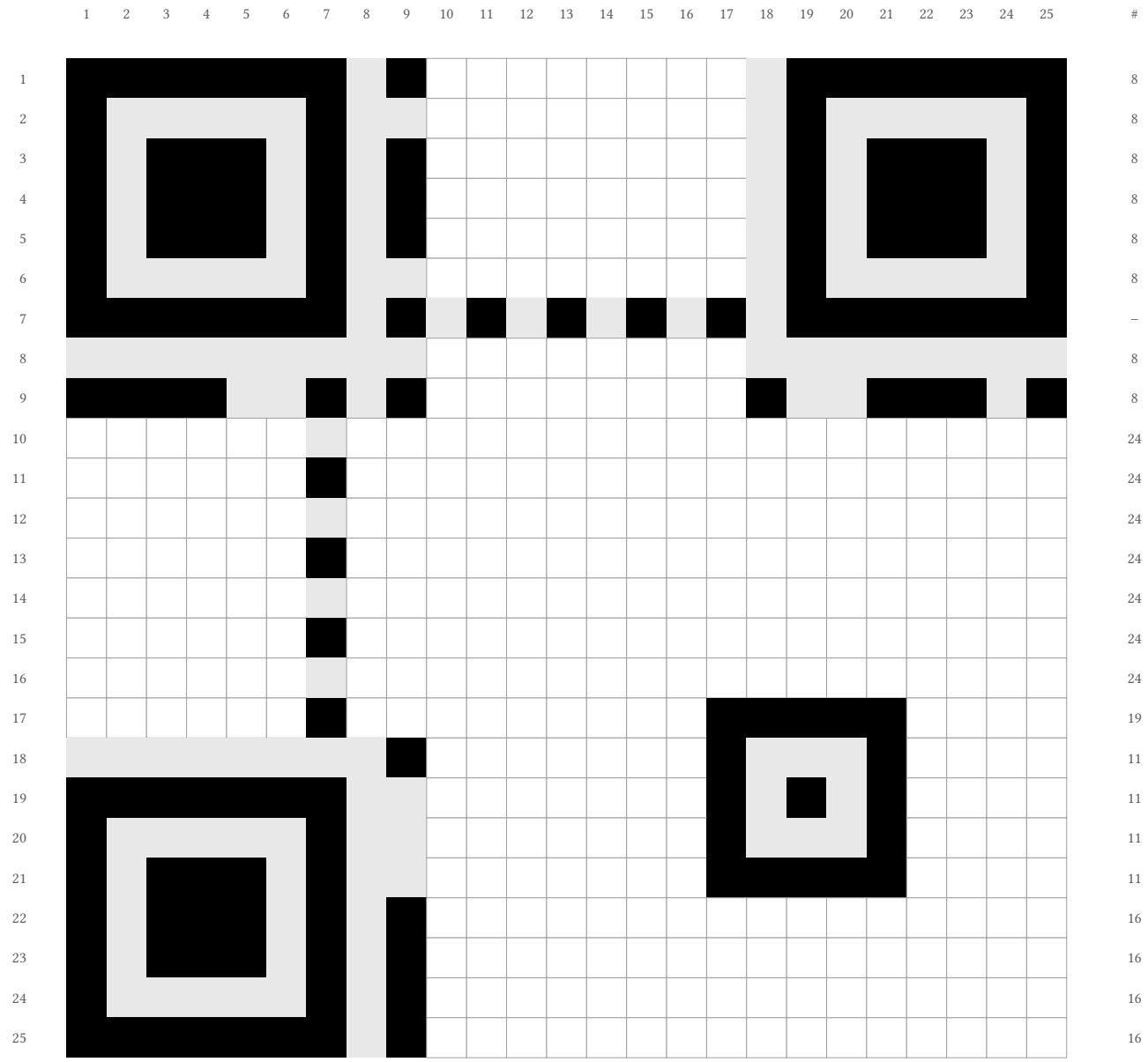
Template 25 × 25 · mask 2

24 words · header `00010` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



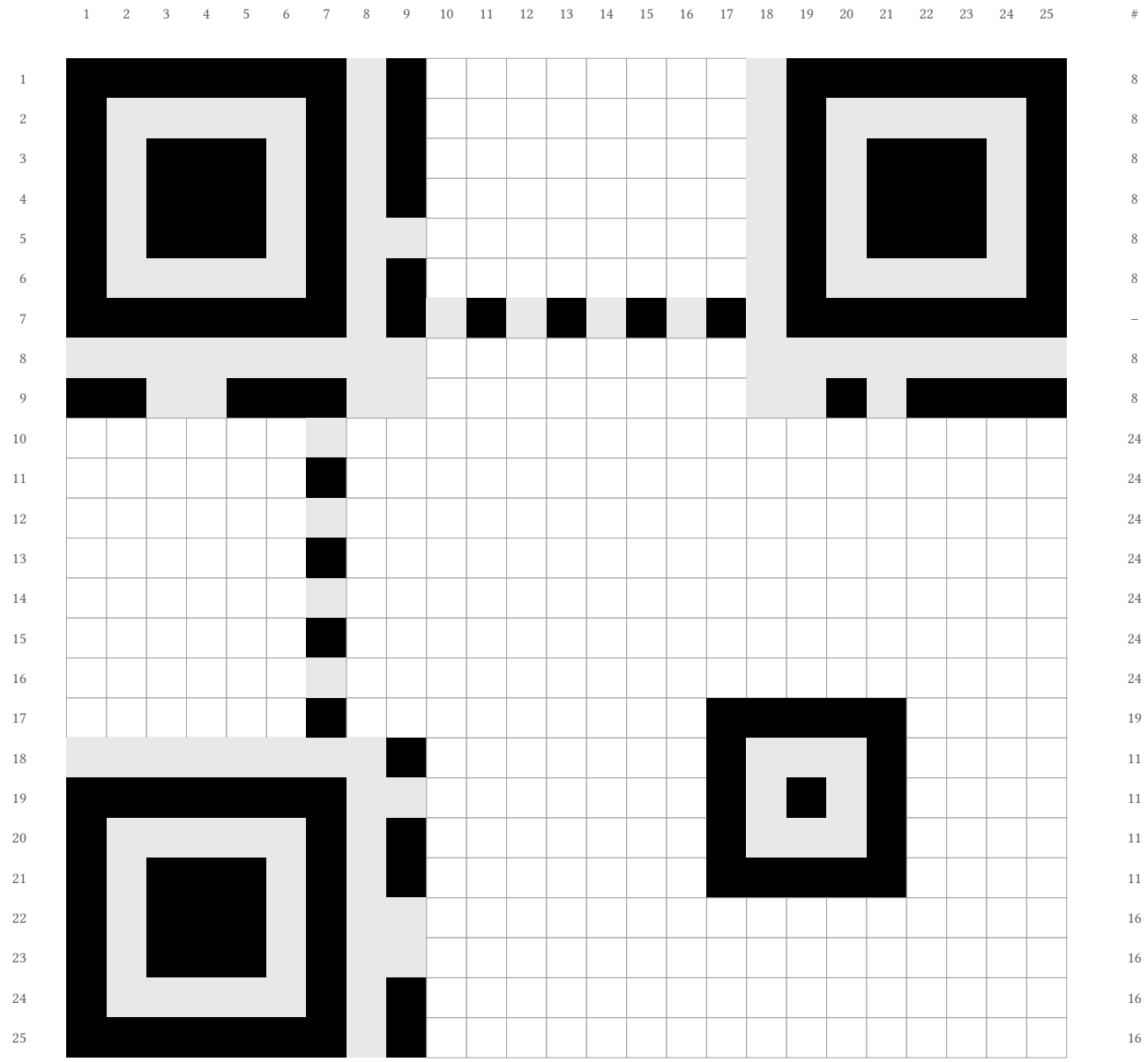
Template 25 × 25 · mask 3

24 words · header `00011` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



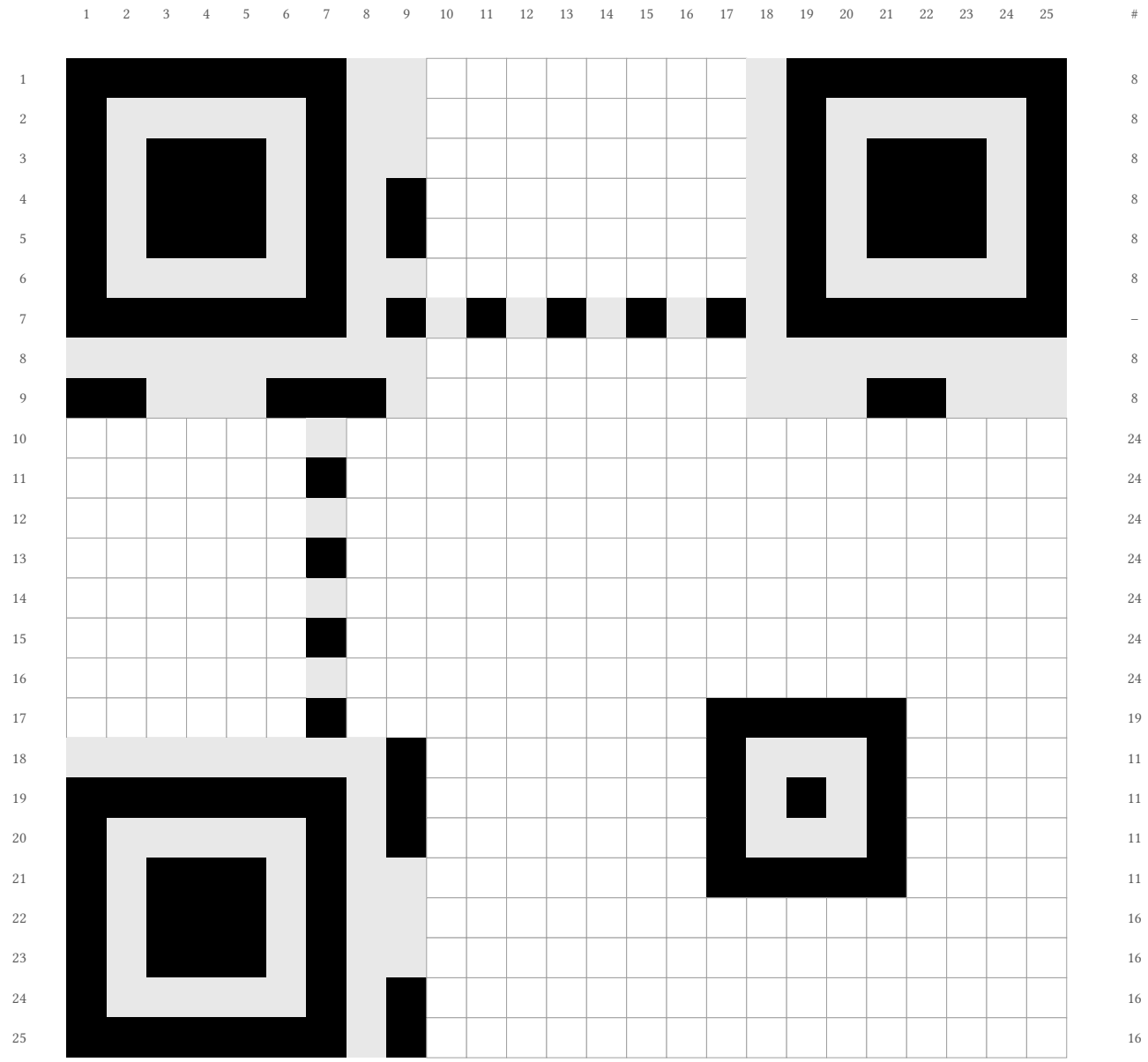
Template 25 × 25 · mask 4

24 words · header `00100` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



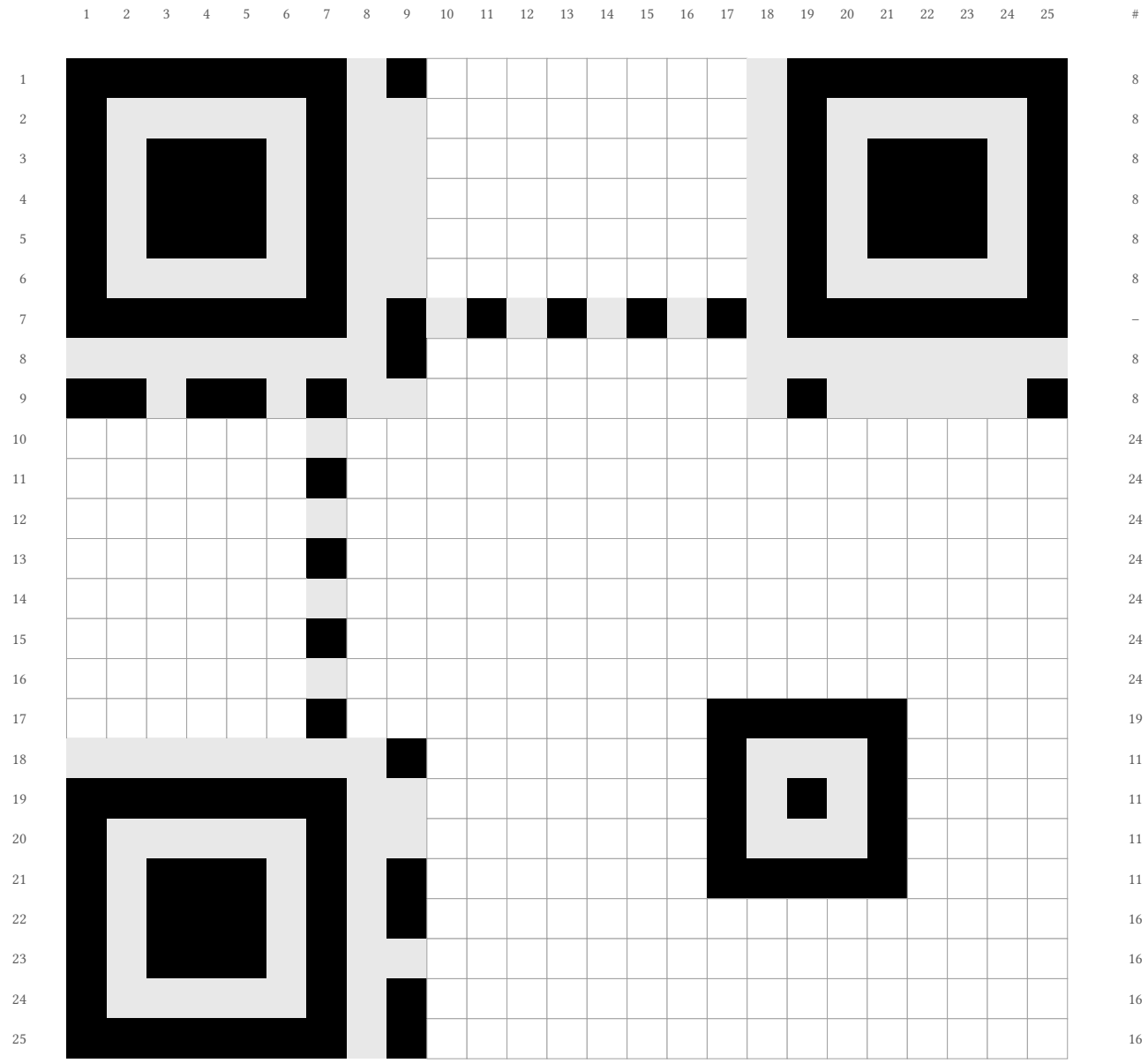
Template 25 × 25 · mask 5

24 words · header `00101` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



Template 25 × 25 · mask 6

24 words · header `00110` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.



Template 25 × 25 · mask 7

24 words · header `00111` · fill the white squares row by row, left to right: 1 = dark, 0 = leave white. Right column: white squares in that row.

